

Computer Vision (EE6102)

Linear and Spatial Filtering

2026 Fall

Linear Image Processing and Filtering

- Math of 2-d linear systems
- Separability
- Shift-invariant linear systems, 2-d convolution
- Integral image for fast box-filtering
- 2-d frequency response
- Blurring and sharpening
- Zoneplate
- Image sampling/aliasing
- Wiener filtering
- Nonlinear noise reduction/sharpening

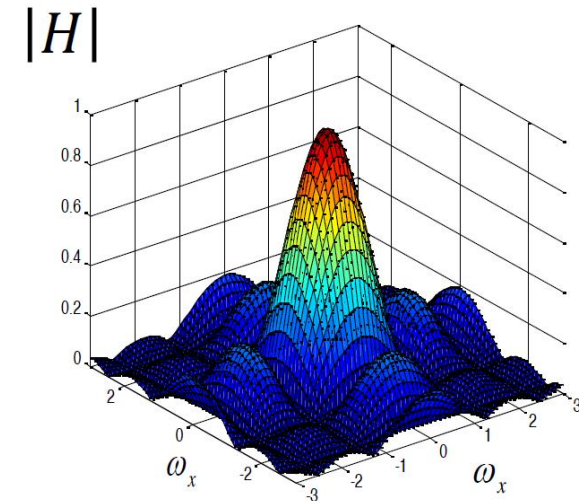
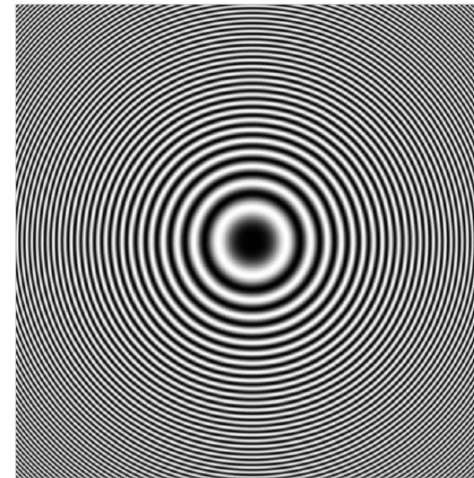
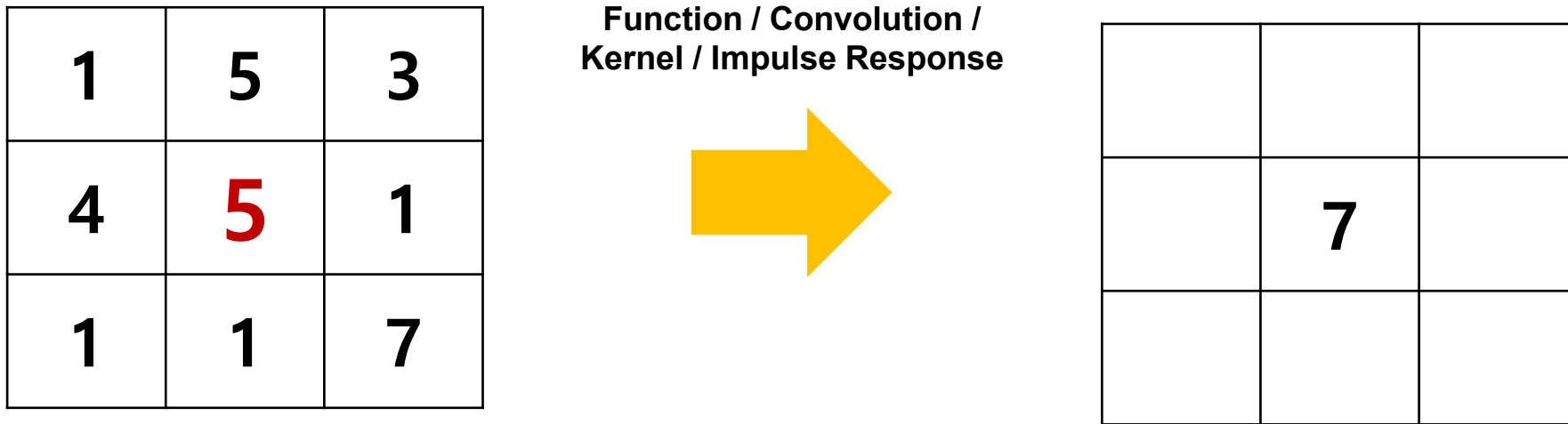


Image Filtering

- Modify the pixels in an image based on some function of a local neighborhood of each pixel



We focus on the center pixel. Its new value is computed from the local neighborhood using a filter function

5 → 7

Linear Filtering (Processing)

- One simple version of filtering: linear filtering (cross-correlation, convolution)
 - Replace each pixel by a linear combination (a weighted sum) of its neighbors
- The prescription for the linear combination is called the “kernel” (or “mask”, “filter”)

1	5	3
4	5	1
1	1	7

Local image data

0	0	0
0	0.5	0
0	1	0.5

Kernel

$$1 \cdot 0 + 5 \cdot 0 + 3 \cdot 0 + 4 \cdot 0 + 5 \cdot 0.5 + 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 1 + 7 \cdot 0.5 = 7$$

	7	

Modified image data

Linear Image Processing

- Image processing system $S(\cdot)$ is linear, if superposition principle holds:

$$S(\alpha \cdot f[x, y] + \beta \cdot g[x, y]) = \alpha \cdot S(f[x, y]) + \beta \cdot S(g[x, y]), \text{ for all } \alpha, \beta \in \mathbb{R}$$

- Any linear image processing system can be written as

$$\mathbf{g} = \mathbf{H}\mathbf{f} \quad \text{Note: matrix } \mathbf{H} \text{ need not to be square}$$

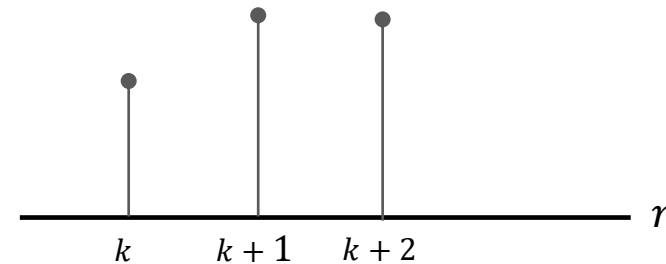
By sorting pixels into a column vector

$$\mathbf{f} = (f[0,0] \ f[1,0] \cdots f[N-1,0] \cdots \cdots f[0,L-1] \ \cdots \ f[N-1,L-1])^T$$

Impulse Response and Convolution

- 1-D discrete signal: Any discrete signal can be represented as a weighted sum of shifted impulses.

$$x[n] = \sum_k x[k] \delta[n - k]$$



- If the impulse response of an LTI system is $h(n)$, then the response to a shifted impulse $\delta(n - k)$ is $h(n - k)$

$$y[n] = \sum_k x[k] h[n - k]$$

$$x[n] = \sum_k x[k] \delta[n - k]$$

$\Downarrow$

$$\delta[n - k] \xrightarrow{\text{LTI system}} h[n - k]$$

$\Downarrow$

$$y[n] = \sum_k x[k] h[n - k] = x[n] * h[n]$$

Note: Convolution is the sum of scaled and shifted impulse response

Impulse Response and Convolution

- Another way to represent any linear image processing scheme

$$g[\alpha, \beta] = \sum_{x=0}^{N-1} \sum_{y=0}^{L-1} f[x, y] \cdot h[x, \alpha, y, \beta]$$

- Input: unit impulse at pixel $[a, b]$

$$f[x, y] = \delta[x - a, y - b] = \begin{cases} 1, & x = a \wedge y = b \\ 0, & \text{else} \end{cases}$$

- Output: impulse response

$$g[\alpha, \beta] = \sum_{x=0}^{N-1} \sum_{y=0}^{L-1} \delta[x - a, y - b] \cdot h[x, \alpha, y, \beta] = h[a, \alpha, b, \beta]$$

Convolution vs Cross-Correlation (FYI)

- Cross-correlation

1. Definitions

Let F be the input image.

Let H be the kernel of size $(2k + 1) \times (2k + 1)$.

Let G be the output image.

2. Cross-correlation formula

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

$$G = H \otimes F$$

Cross-correlation operator

3. Conceptual summary

- At each pixel (i, j) , place the kernel on the local image neighborhood.
- Multiply corresponding entries and sum them.
- This is a dot product between the kernel and the local image patch.
- The kernel is **NOT** flipped in cross-correlation.

- Cross-correlation vs Convolution

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

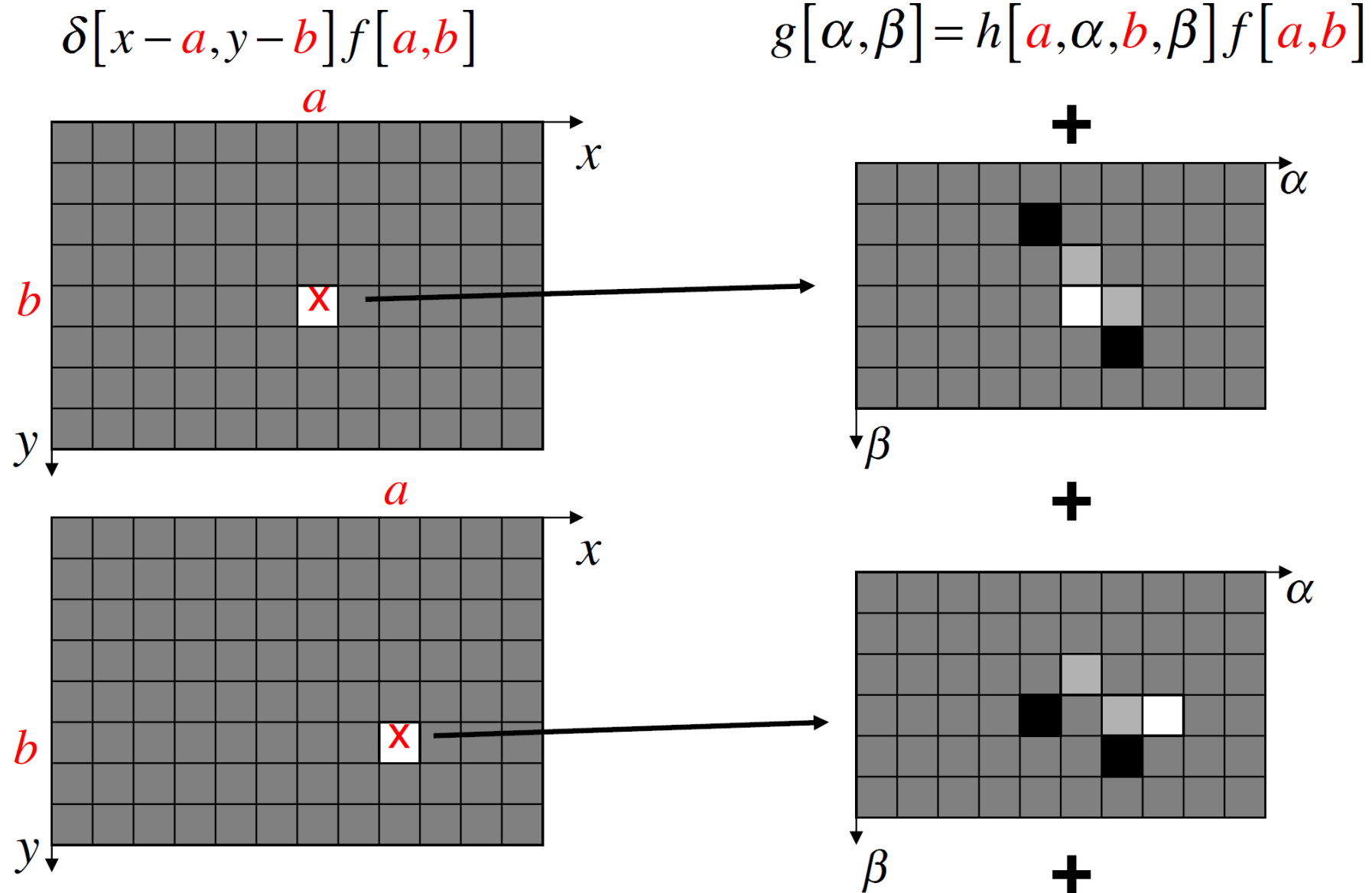
Kernel non-flipped



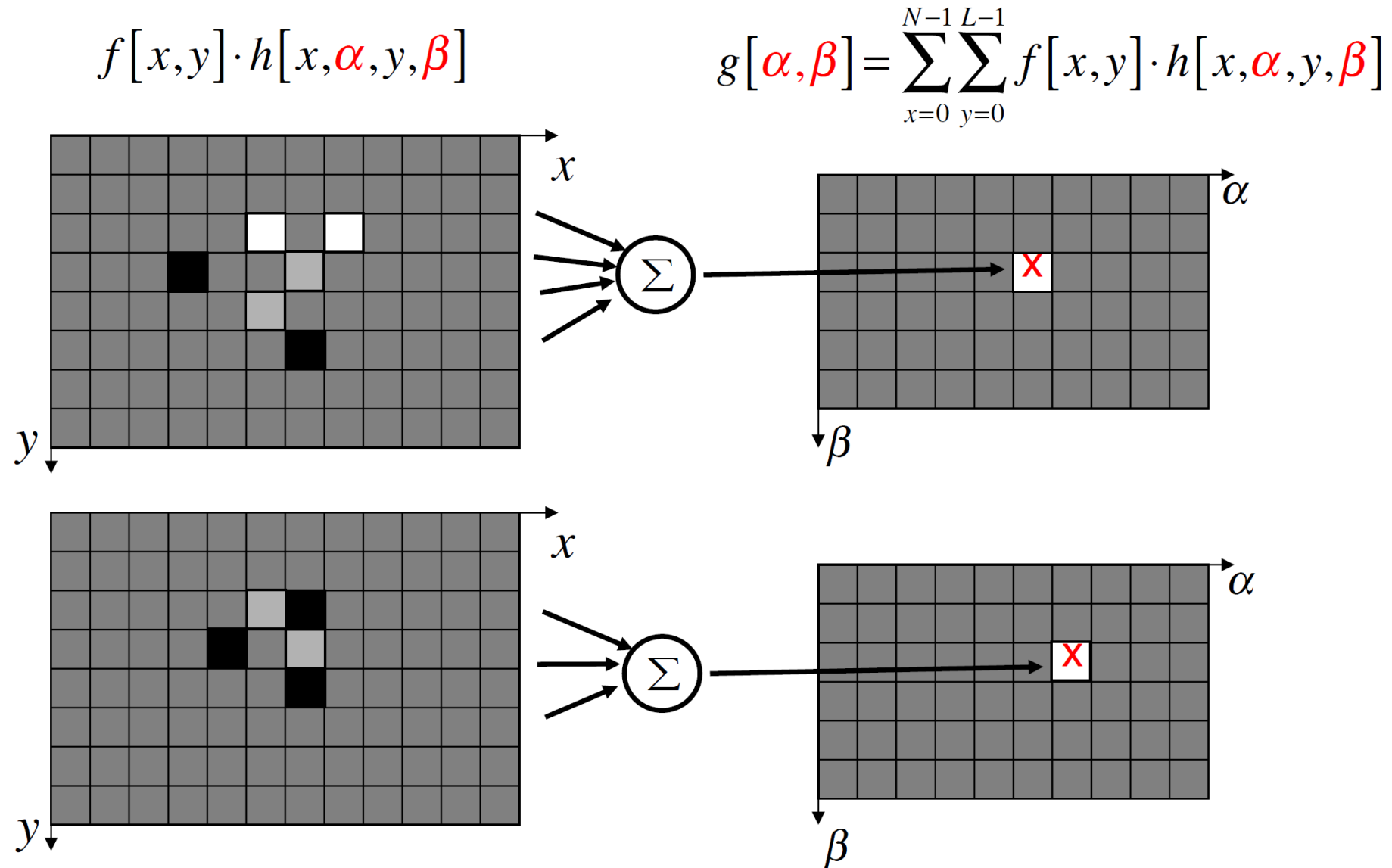
$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

Kernel flipped

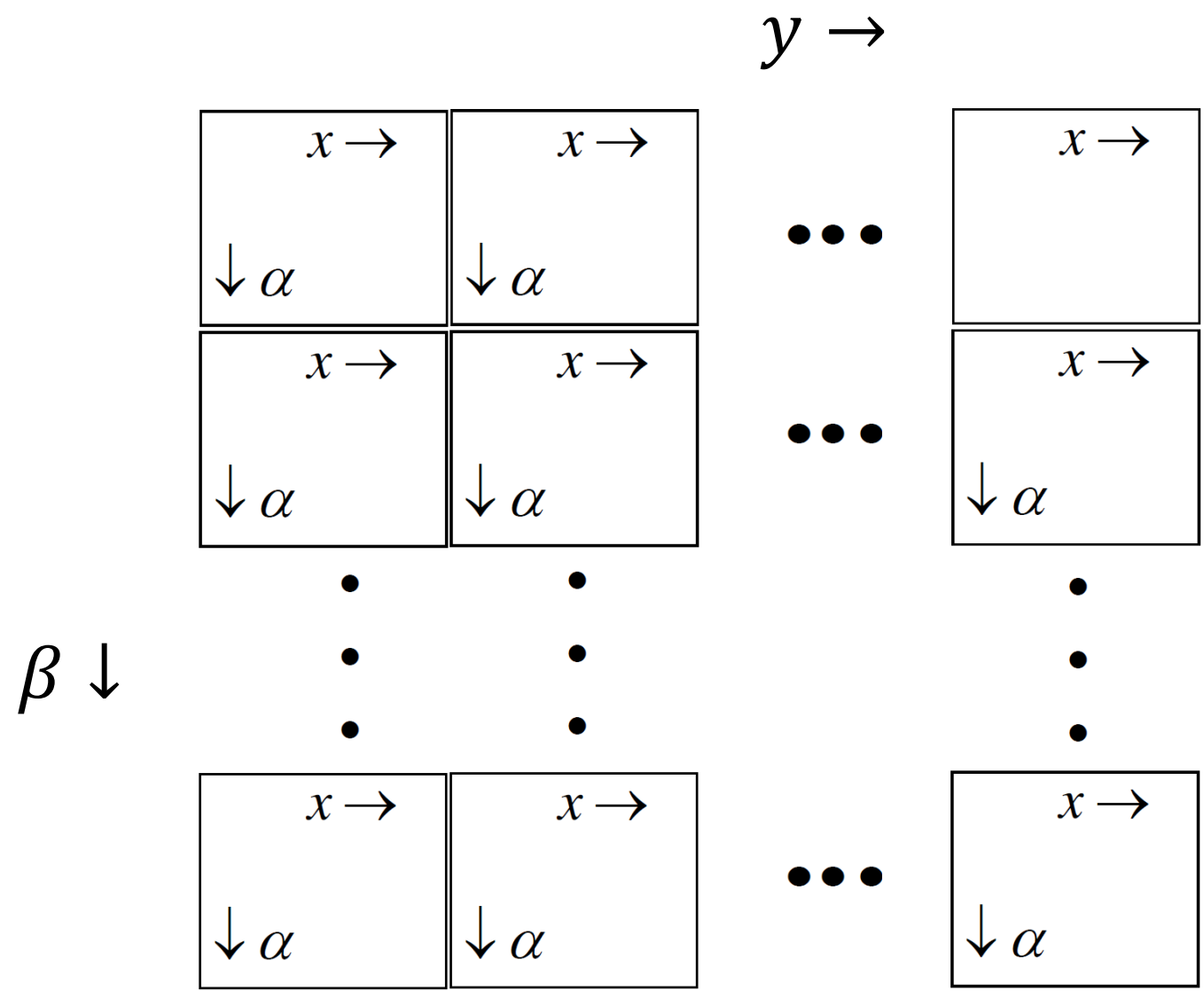
Interpretation #1: superposition of impulse responses



Interpretation #2: superposition of impulse responses



Relationship of H and $h[x, \alpha, y, \beta]$



Separable Linear Image Processing

- Impulse response is separable in $[x, \alpha]$ and $[y, \beta]$ i.e., can be written as

$$g[\alpha, \beta] = \sum_{x=0}^{N-1} \sum_{y=0}^{L-1} f[x, y] \cdot h_x[x, \alpha] \cdot h_y[y, \beta]$$

- Processing can be performed
 - Row by row, the column by column

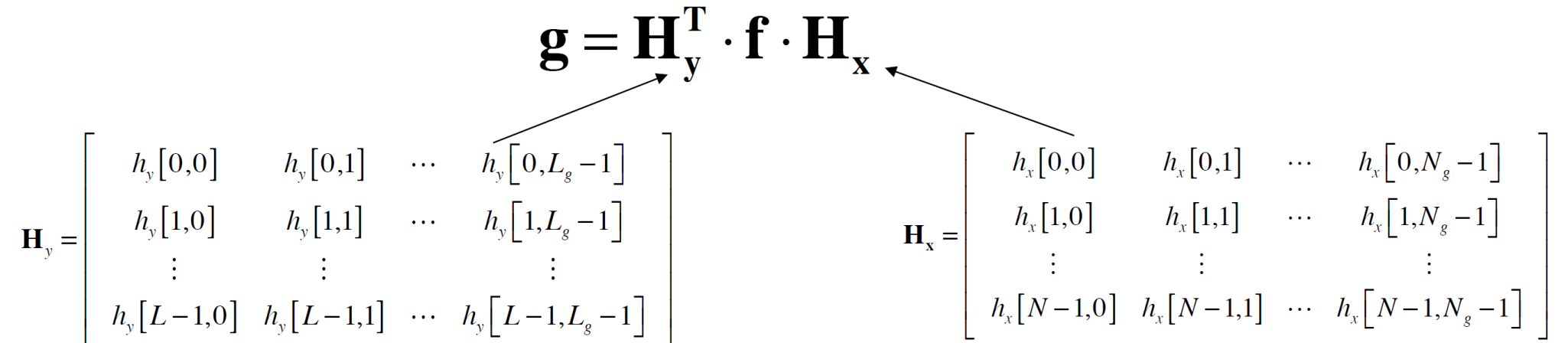
$$g[\alpha, \beta] = \sum_{x=0}^{N-1} h_y[y, \beta] \sum_{y=0}^{L-1} f[x, y] \cdot h_x[x, \alpha]$$

- Column by column, the row by row

$$g[\alpha, \beta] = \sum_{x=0}^{N-1} h_x[x, \alpha] \sum_{y=0}^{L-1} f[x, y] \cdot h_y[y, \alpha]$$

Separable Linear Image Processing

- If the digital input and output images are written as a matrices $\mathbf{f}$ and $\mathbf{g}$, we can conveniently write

$$\mathbf{g} = \mathbf{H}_y^T \cdot \mathbf{f} \cdot \mathbf{H}_x$$

$$\mathbf{H}_y = \begin{bmatrix} h_y[0,0] & h_y[0,1] & \cdots & h_y[0,L_g-1] \\ h_y[1,0] & h_y[1,1] & \cdots & h_y[1,L_g-1] \\ \vdots & \vdots & & \vdots \\ h_y[L-1,0] & h_y[L-1,1] & \cdots & h_y[L-1,L_g-1] \end{bmatrix}$$
$$\mathbf{H}_x = \begin{bmatrix} h_x[0,0] & h_x[0,1] & \cdots & h_x[0,N_g-1] \\ h_x[1,0] & h_x[1,1] & \cdots & h_x[1,N_g-1] \\ \vdots & \vdots & & \vdots \\ h_x[N-1,0] & h_x[N-1,1] & \cdots & h_x[N-1,N_g-1] \end{bmatrix}$$

- Output image $\mathbf{g}$ has size $L_g \times N_g$
- If the operator does not change image size, $\mathbf{H}_x$ and $\mathbf{H}_y$ are square matrices

Example: Separable Sobel Kernel

1. General separable 2-D linear processing (matrix form)

For an image $f \in \mathbb{R}^{L \times N}$, a separable linear operator can be written as two successive 1-D operations:

$$\mathbf{g} = \mathbf{H}_y^T \mathbf{f} \mathbf{H}_x$$

- $\mathbf{H}_x \in \mathbb{R}^{N \times N}$: 1-D operator along x (each row)
- $\mathbf{H}_y \in \mathbb{R}^{L \times L}$: 1-D operator along y (each column)
- If the image size does not change (e.g., with padding), $\mathbf{H}_x$ and $\mathbf{H}_y$ are square matrices.

2. Relation to 1-D kernels

In practice, $\mathbf{H}_x$ and $\mathbf{H}_y$ are constructed from 1-D kernels h_x and h_y by sliding them across the entire image.

1-D kernels (small)

$$h_x = [h_x[-r], \dots, h_x[r]]$$

$$h_y = [h_y[-s], \dots, h_y[s]]^T$$



Full operator matrices (large)

$$h_x \rightarrow \mathbf{H}_x \in \mathbb{R}^{N \times N}$$

$$h_y \rightarrow \mathbf{H}_y \in \mathbb{R}^{L \times L}$$

(Toeplitz structure, determined by boundary condition)

3. Separable 2-D kernel (convolution form)

If the system is shift-invariant, a separable 2-D convolution kernel can be written as the outer product of two 1-D kernels:

$$k[m, n] = h_y[m] h_x[n]$$

- $k[m, n]$: 2-D kernel (small, e.g., 3×3)
- $h_x[n]$: horizontal 1-D kernel
- $h_y[m]$: vertical 1-D kernel
- Applying $k[m, n]$ to an image is equivalent to two successive 1-D convolutions: first with h_x (along x), then with h_y (along y).

4. Sobel example (kernels)

1-D kernels

(horizontal derivative and vertical smoothing)

$$h_x = [-1 \ 0 \ 1]$$

(horizontal derivative)

$$h_y = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

(vertical smoothing)

2-D Sobel kernel (horizontal gradient)

$$k_x[m, n] = h_y[m] h_x[n]$$

$$k_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Note: We use k_x for the 2-D kernel.
(Do not confuse with the operator matrix $\mathbf{H}_x$).

5. Image filtering results

Original image



2-D Sobel (one-step)

$$g_1 = f * k_x$$



Two 1-D filters (separable)

$$g_2 = (f * h_x) * h_y$$



Difference

$$|g_1 - g_2|$$



Max difference: 0
(identical results)

6. Takeaways

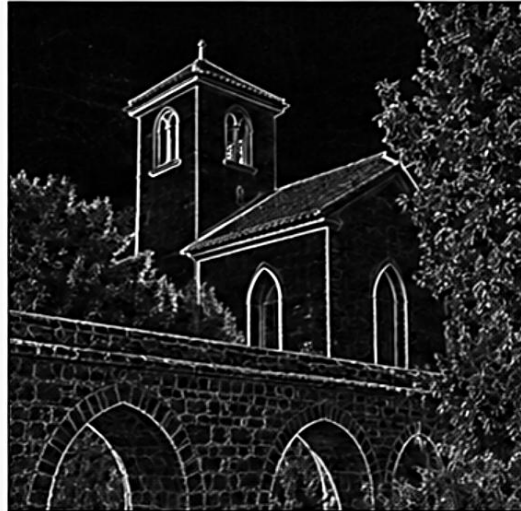
- General separable processing: $\mathbf{g} = \mathbf{H}_y^T \mathbf{f} \mathbf{H}_x$ (matrix form).
- Sobel kernel is separable: $k_x = h_y h_x$ with $h_x = [-1, 0, 1]$, $h_y = [1, 2, 1]^T$.
- Convolution with the 2-D Sobel kernel gives the same result as two successive 1-D convolutions (horizontal derivative, then vertical smoothing).
- A separable 2-D kernel reduces computation (for $K \times K$: $K^2 \rightarrow 2K$ multiplications per pixel) without changing the result.
- **Keep notation consistent: use k , h for small kernels, and H_x , H_y for full operator matrices.**

Example: Separable Sobel Kernel

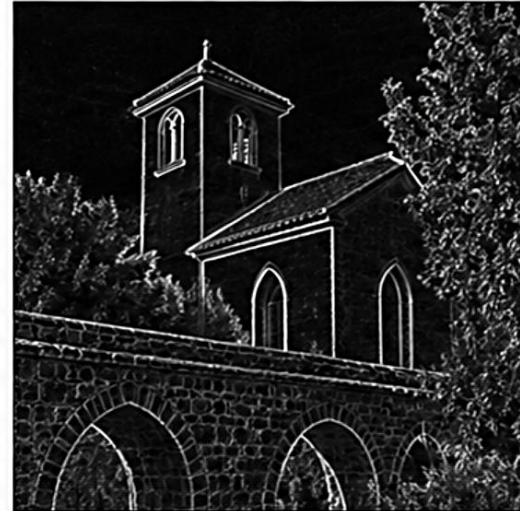
Original



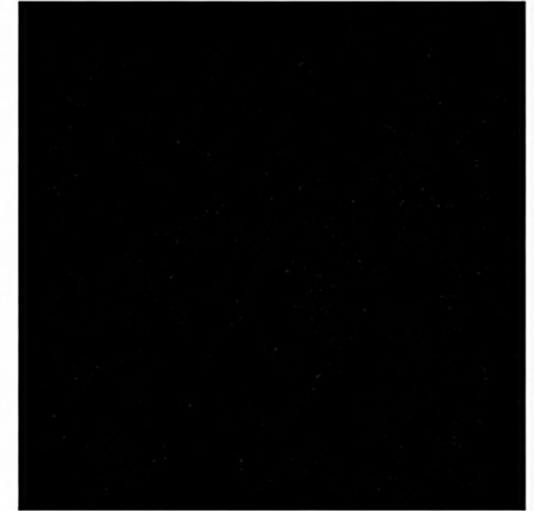
2-D Sobel
(single 3×3 convolution)



Separable Sobel
(1×3 then 3×1)



Difference ≈ 0



Sobel x kernel

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

=

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}^T \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

A separable 2-D kernel can be implemented as two successive 1-D convolutions with same final result

Example: Subsampling

- Image subsampling 2:1 horizontally and vertically

Input (8×8)

F

f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}	f_{17}	f_{18}
f_{21}	f_{22}	f_{23}	f_{24}	f_{25}	f_{26}	f_{27}	f_{28}
f_{31}	f_{32}	f_{33}	f_{34}	f_{35}	f_{36}	f_{37}	f_{38}
f_{41}	f_{42}	f_{43}	f_{44}	f_{45}	f_{46}	f_{47}	f_{48}
f_{51}	f_{52}	f_{53}	f_{54}	f_{55}	f_{56}	f_{57}	f_{58}
f_{61}	f_{62}	f_{63}	f_{64}	f_{65}	f_{66}	f_{67}	f_{68}
f_{71}	f_{72}	f_{73}	f_{74}	f_{75}	f_{76}	f_{77}	f_{78}
f_{81}	f_{82}	f_{83}	f_{84}	f_{85}	f_{86}	f_{87}	f_{88}

Horizontal 2:1
subsampling

Vertical 2:1
subsampling

g_{11}	g_{12}	g_{13}	g_{14}
g_{21}	g_{22}	g_{23}	g_{24}
g_{31}	g_{32}	g_{33}	g_{34}
g_{41}	g_{42}	g_{43}	g_{44}

G

Output (4×4)

1-D subsampling matrix

$$H_x = H_y = H$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (8 \times 4)$$

Maps 8 samples to 4 samples
(when transposed/applied approximately)

$$G = H_y^T F H_x$$

Apply 1-D subsampling independently along x and y

Summary

- Separable linear processing
- Horizontal 1-D subsampling + vertical 1-D subsampling
- Same 2-D result without constructing a large 2-D operator

Note: Subsampling is a linear operation, but it is not a shift-invariant convolution filter

Example: Subsampling

- A somewhat better technique for 2:1 image size reduction

$$\mathbf{H}_x = \mathbf{H}_y = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0.5 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$$



$$\mathbf{H}_x = \mathbf{H}_y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\frac{x_0 + x_1}{2}, \quad \frac{x_2 + x_3}{2}, \quad \frac{x_4 + x_5}{2}, \quad \frac{x_6 + x_7}{2}$$

$$[x_0, x_1, x_2, x_3, \dots] \rightarrow [x_0, x_2, x_4, x_6, \dots]$$

Example: Filtering

- Each pixel is replaced by the average of two horizontally (vertically) neighboring pixels

$$\mathbf{H}_x = \mathbf{H}_y = \begin{bmatrix} 0.5 & 0 & & \dots & & & 0 \\ 0.5 & 0.5 & & & & & \\ 0 & 0.5 & 0.5 & & & & \\ & & 0.5 & 0.5 & \ddots & & \vdots \\ & & & 0.5 & 0.5 & & \\ \vdots & & & \ddots & 0.5 & 0.5 & \\ & & & & 0.5 & 0.5 & 0 \\ 0 & & \dots & & 0 & 0.5 & 0.5 \end{bmatrix}$$

$$[x_0, x_1, x_2, x_3, \dots] \rightarrow \left[\frac{x_0 + x_1}{2}, \frac{x_1 + x_2}{2}, \frac{x_2 + x_3}{2}, \dots \right]$$

$$\text{In 2D, } \mathbf{g} = \mathbf{H}_y^T \mathbf{f} \mathbf{H}_x$$

$$\frac{1}{4} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

(2x2 box smoothing)

- How this kernel can be performed as a moving average filter (Smoothing operator)
- Shift-invariant operation (except for image boundary)

Shift-invariant systems and Toeplitz matrices

- For a separable, shift-invariant, linear system

$$h_x[x, \alpha] = h_{siv/x}[\alpha - x] \quad \text{and} \quad h_y[y, \beta] = h_{siv/y}[\beta - y]$$

- Matrices $\mathbf{H}_x$ and $\mathbf{H}_y$ are square, and Toeplitz matrices

$$\mathbf{H}_x = \begin{bmatrix} h_{siv/x}[0] & h_{siv/x}[1] & \cdots & h_{siv/x}[N-1] \\ h_{siv/x}[-1] & h_{siv/x}[0] & \cdots & h_{siv/x}[N-2] \\ \vdots & \vdots & & \vdots \\ h_{siv/x}[1-N] & h_{siv/x}[2-N] & \cdots & h_{siv/x}[0] \end{bmatrix}$$

- Operation is a 2-D separable convolution ('filtering')

$$g[\alpha, \beta] = \sum_{x=0}^{N-1} \sum_{y=0}^{L-1} f[x, y] \cdot h_{siv/x}[\alpha - x][\beta - y]$$

Non-separable 2-d convolution

- Convolution kernel of linear shift-invariant system („filter“) can also be non-separable

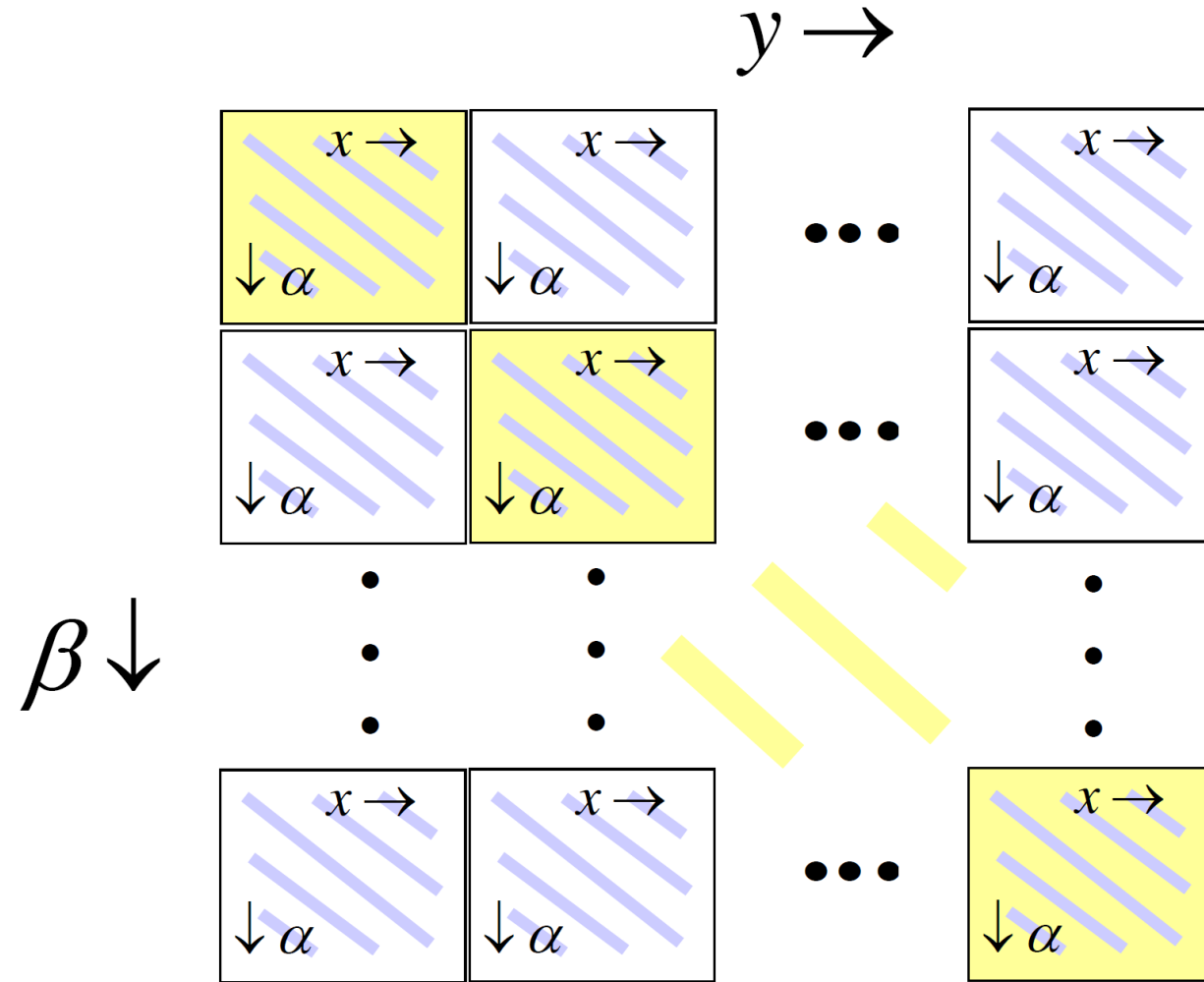
$$g[\alpha, \beta] = \sum_{x=0}^{N-1} \sum_{y=0}^{L-1} f[x, y] \cdot h_{siv}[\alpha - x, \beta - y]$$

- View as a matrix operation

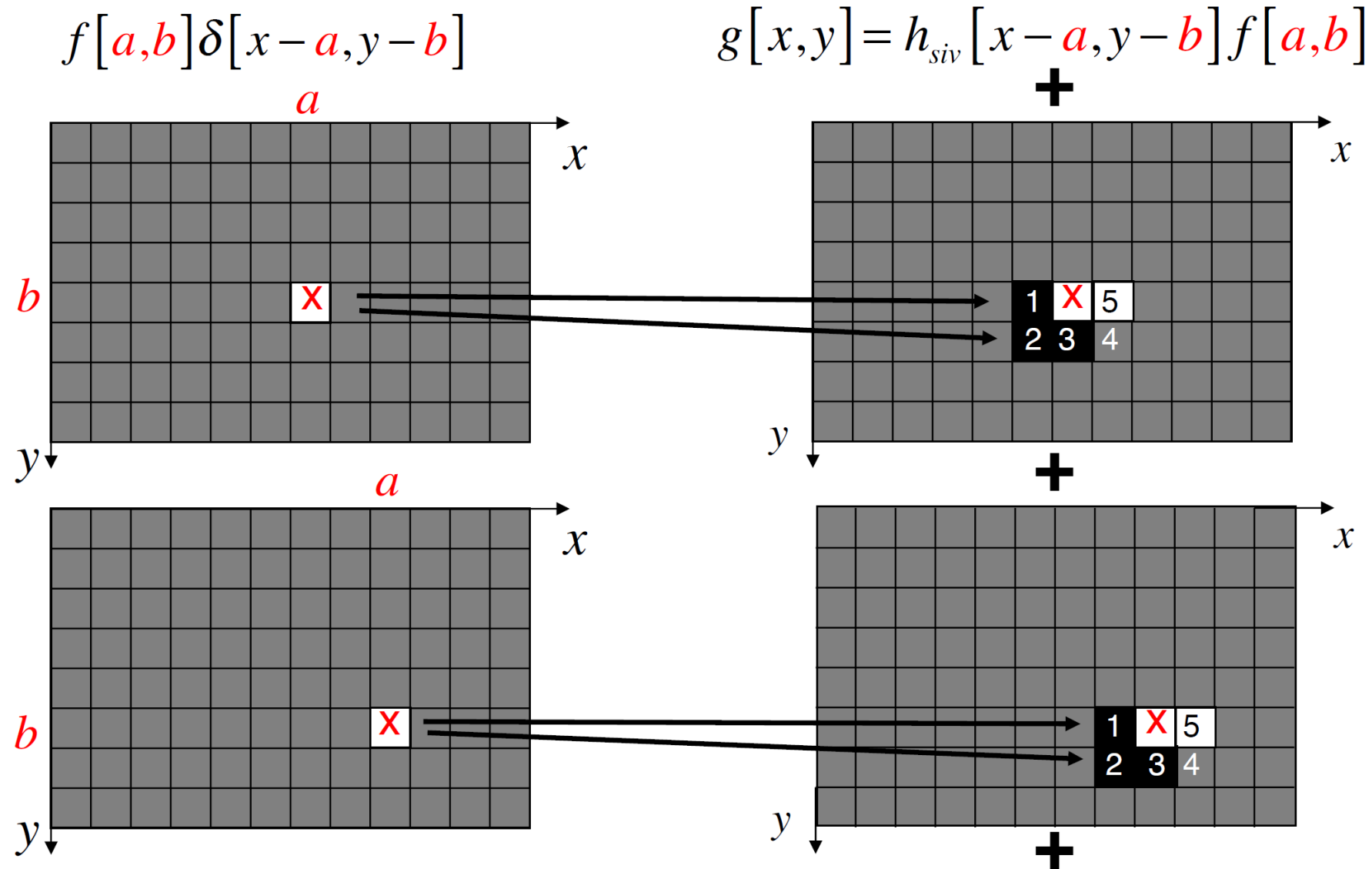
$$\mathbf{g} = \mathbf{H}\mathbf{f}$$

$\mathbf{H}$ is a block Toeplitz matrix

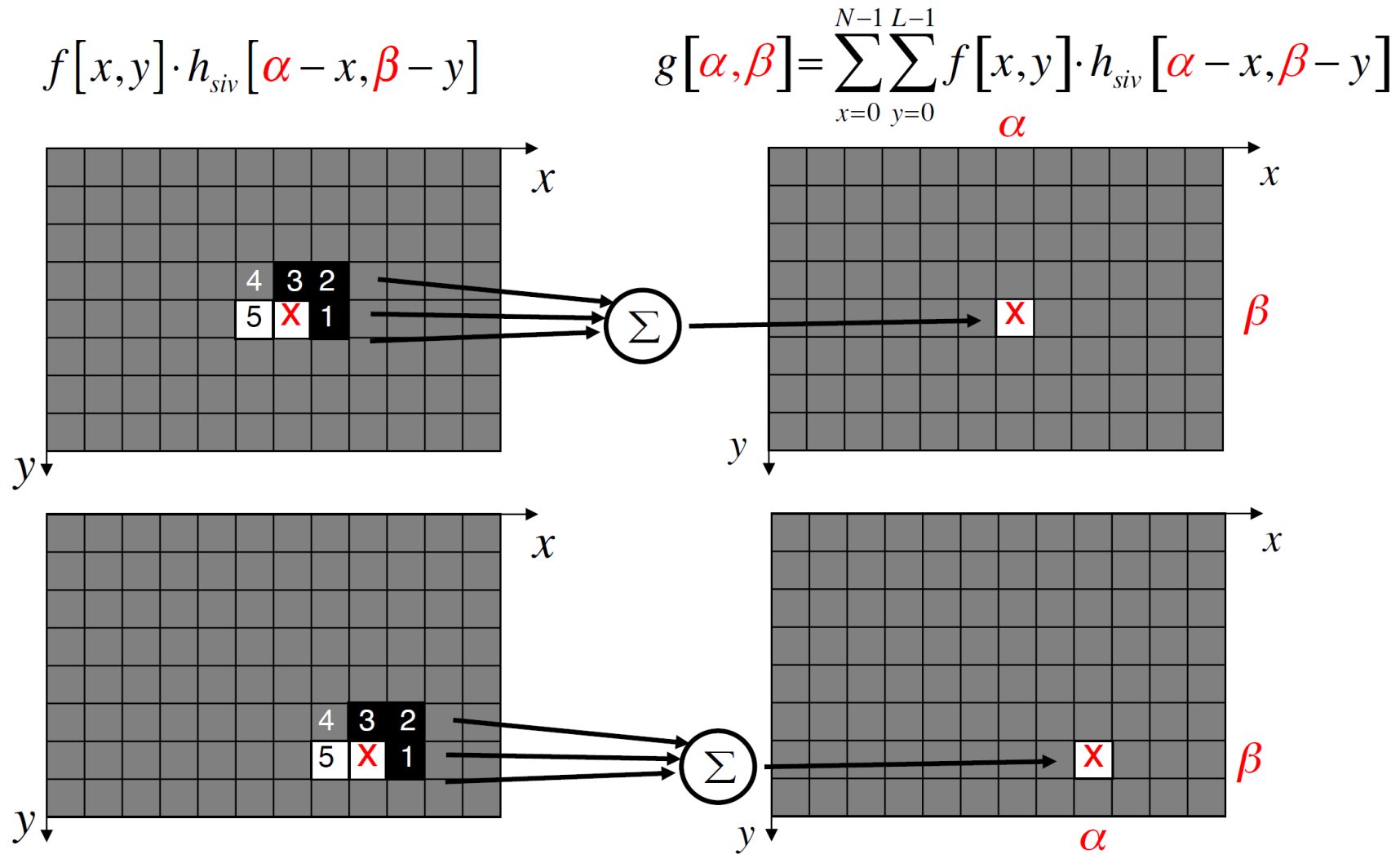
Structure of H for non-separable convolution



Convolution: superposition of impulse responses



Convolution: linear combination of neighboring pixel values



Convolution Examples



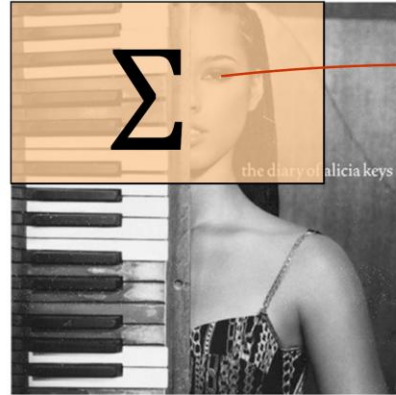
Original



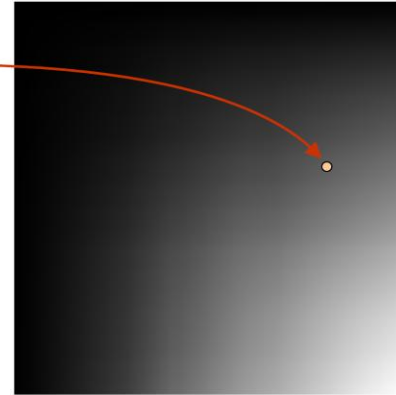
Blurred by convolution impulse response

$$\frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \frac{1}{5} [1 \quad 1 \quad 1 \quad 1 \quad 1] + \frac{1}{5} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Box Filtering with Integral Image

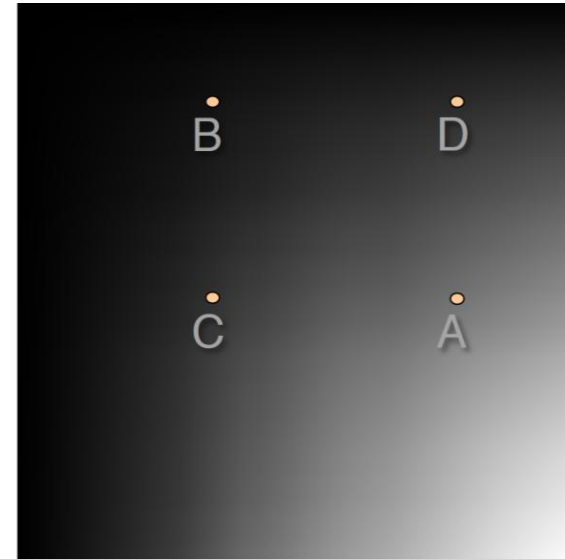
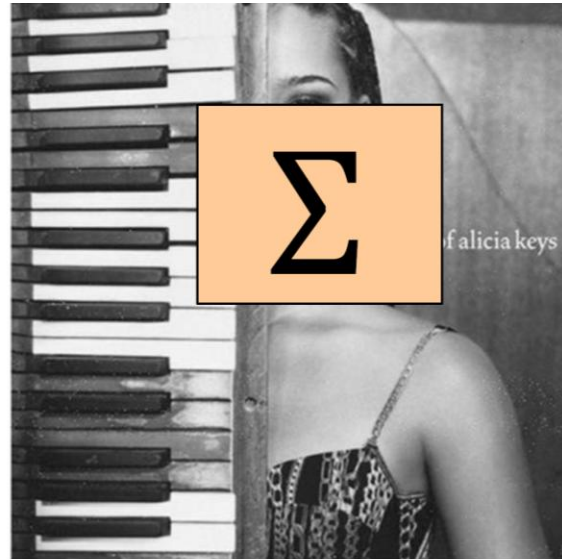


$f[x,y]$



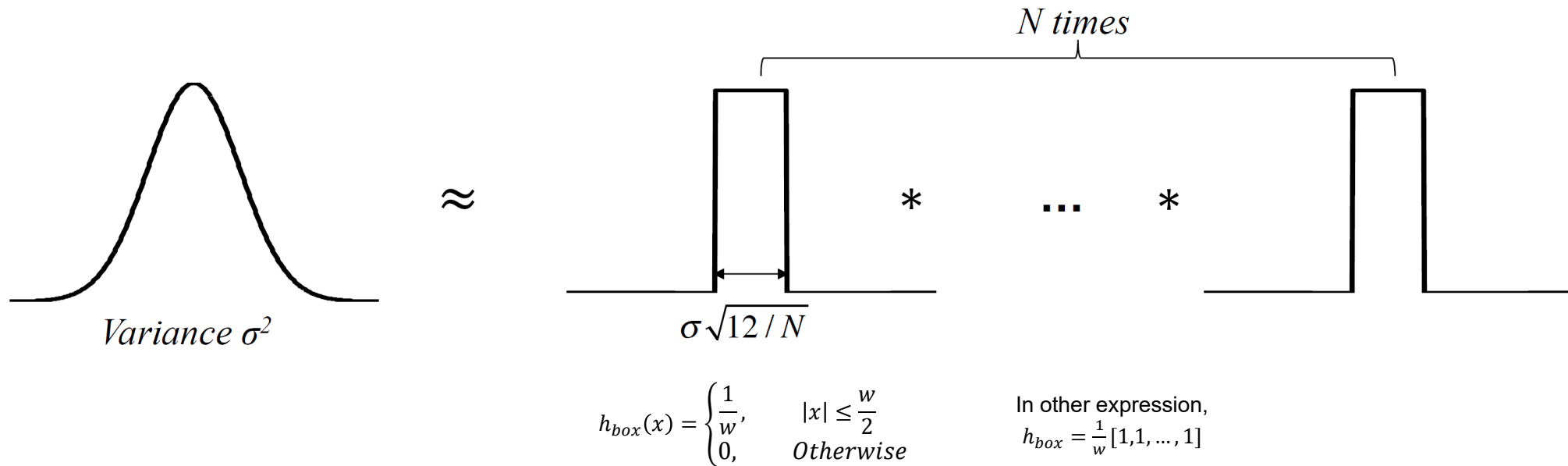
$$i[x,y] = \sum_{u=0}^x \sum_{v=0}^y f[u,v]$$

$$\begin{aligned} j[x,y] &= j[x,y-1] + f[x,y] \\ i[x,y] &= i[x-1,y] + j[x,y] \end{aligned}$$



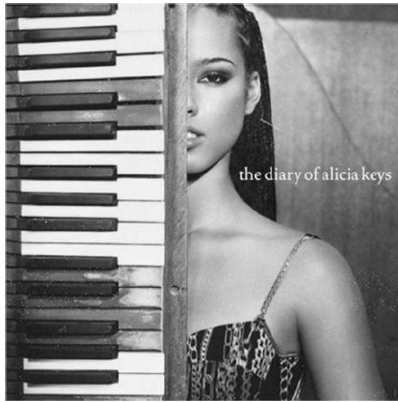
$$\text{Sum} = A + B - C - D$$

Gaussian Filtering by Repeated Box Filtering

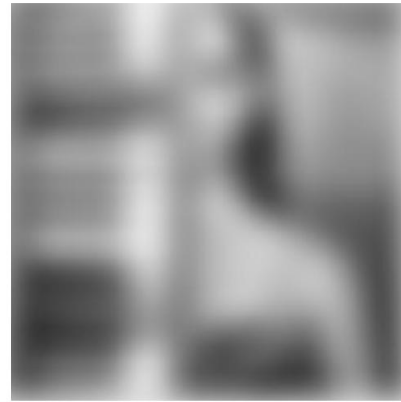


- Approximate convolution with Gaussian kernel by convolution with box kernel, repeated N times
- Convolution with box kernel can be computed efficiently using integral image

Gaussian Filtering by Repeated Box Filtering



Original image
500x500



Gaussian filtered
 $\sigma = 20$, 81x81 kernel

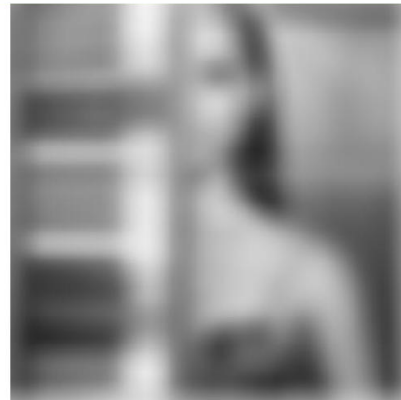
Direct implementation:
6561 multiplications and
6560 additions per pixel

x-y separable filtering:
162 multiplications and
160 additions per pixel

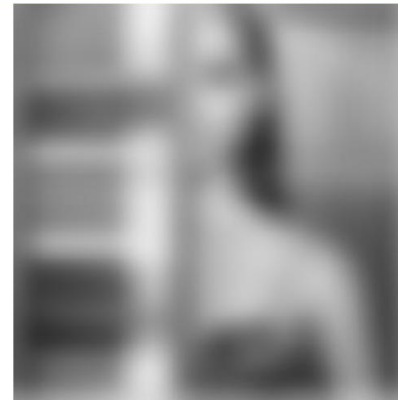
20 additions or subtractions per pixel



Box filtered
after $N = 1$



Box filtered
after $N = 2$

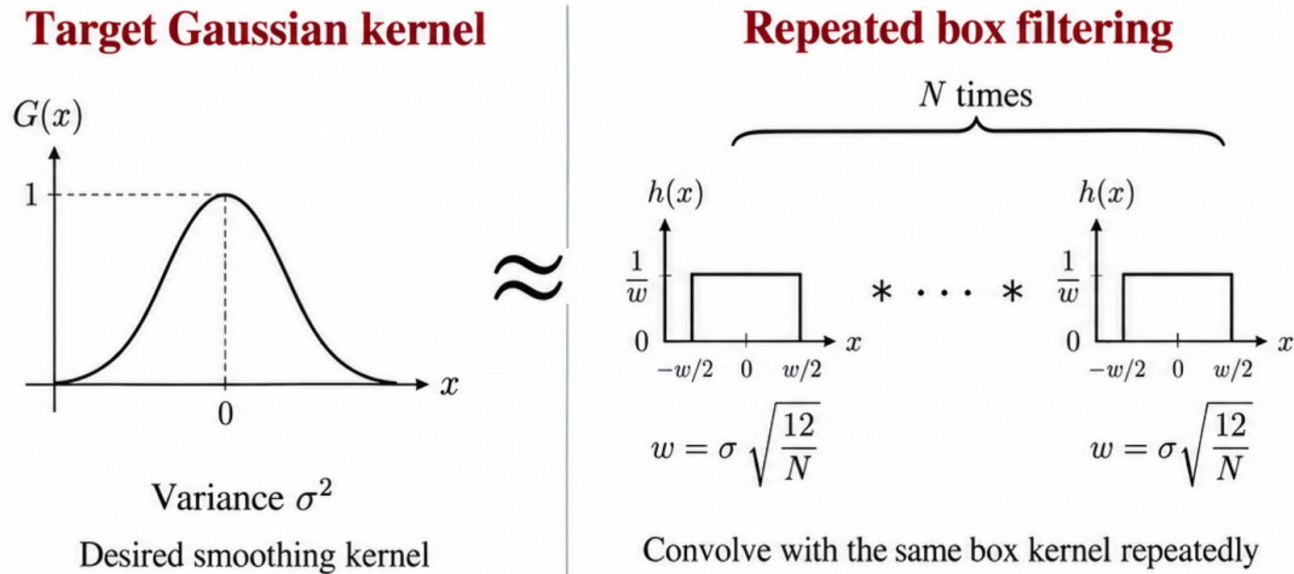


Box filtered
after $N = 3$



Box filtered
after $N = 4$

Gaussian Filtering by Repeated Box Filtering



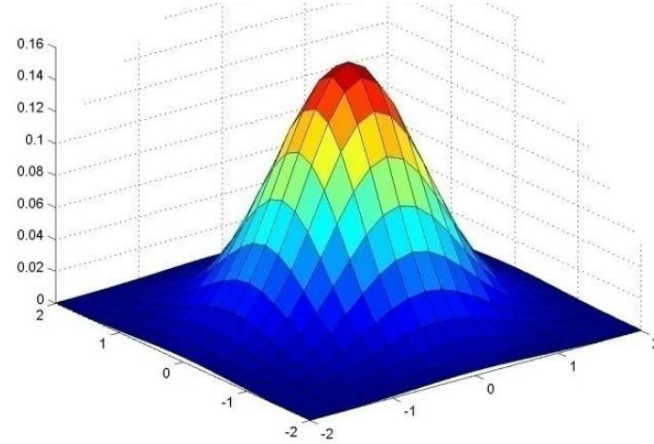
Conceptual Summary

- Separable smoothing operation
- A Gaussian can be approximated by repeated box filtering
- As N increase, the result become closer to Gaussian
- Each box filter can be computed efficiently using an integral image

Gaussian $\approx$ Box · Box · Box · Box (repeated N times)

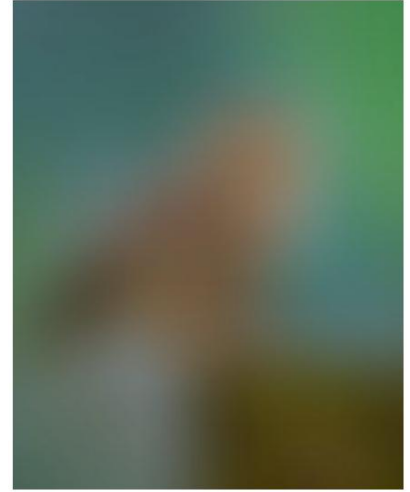
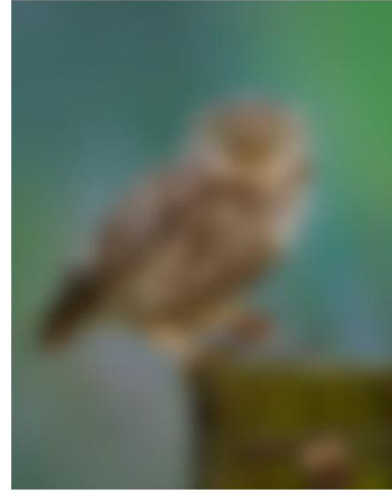
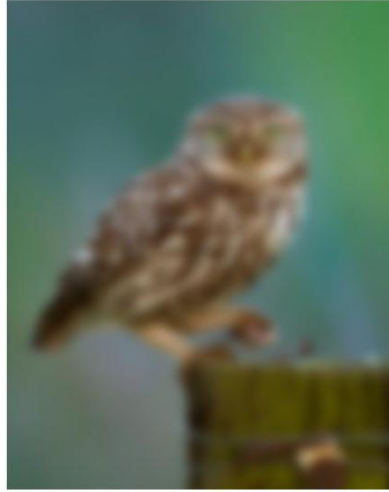
Note: Fast box filtering enables efficient approximation of Gaussian smoothing

Gaussian Filtering



$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

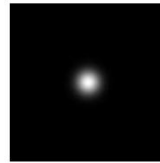
Gaussian Filtering



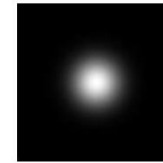
$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



$$\sigma = 1$$



$$\sigma = 5$$



$$\sigma = 10$$



$$\sigma = 30$$

$$K_{\sigma=1} \approx \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

$$K_{\sigma=5} \approx \frac{1}{2308} \begin{bmatrix} 85 & 90 & 92 & 90 & 85 \\ 90 & 96 & 98 & 96 & 90 \\ 92 & 98 & 100 & 98 & 92 \\ 90 & 96 & 98 & 96 & 90 \\ 85 & 90 & 92 & 90 & 85 \end{bmatrix}$$

Gaussian Filtering – How to generate the Gaussian kernel (FYI)

To construct a 5×5 Gaussian kernel with $\sigma = 5$, we start from the Gaussian function

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right).$$

Since $\sigma = 5$ and the kernel size is 5×5 , we use the coordinates

$$x, y \in \{-2, -1, 0, 1, 2\}.$$

At the center,

$$G(0, 0) = 1.$$

For example, at the neighboring position $(1, 0)$,

$$G(1, 0) = \exp\left(-\frac{1}{2 \cdot 25}\right) = e^{-0.02} \approx 0.9802.$$

At the diagonal position $(1, 1)$,

$$G(1, 1) = \exp\left(-\frac{2}{50}\right) = e^{-0.04} \approx 0.9608.$$

For $(2, 0)$,

$$G(2, 0) = \exp\left(-\frac{4}{50}\right) = e^{-0.08} \approx 0.9231.$$

At the outer corner $(2, 2)$,

$$G(2, 2) = \exp\left(-\frac{8}{50}\right) = e^{-0.16} \approx 0.8521.$$

Therefore, the unnormalized 5×5 Gaussian kernel is approximately

$$\begin{bmatrix} 0.8521 & 0.9048 & 0.9231 & 0.9048 & 0.8521 \\ 0.9048 & 0.9608 & 0.9802 & 0.9608 & 0.9048 \\ 0.9231 & 0.9802 & 1.0000 & 0.9802 & 0.9231 \\ 0.9048 & 0.9608 & 0.9802 & 0.9608 & 0.9048 \\ 0.8521 & 0.9048 & 0.9231 & 0.9048 & 0.8521 \end{bmatrix}.$$

If needed, the kernel can then be normalized so that the sum of all coefficients becomes 1.

Is Gaussian Filter Separable?

- A 2-D Gaussian is separable — regardless of kernel size

$$\begin{aligned} G(x, y) &= \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \\ &= \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/(2\sigma^2)}}_{g(x)} \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-y^2/(2\sigma^2)}}_{g(y)} \\ \boxed{G(x, y) = g(x)g(y)} \end{aligned}$$

- Example: 11x11 size

$$\mathbf{g} = [g_{-5} \quad g_{-4} \quad \cdots \quad g_0 \quad \cdots \quad g_5]^T$$

$$\boxed{\mathbf{G} = \mathbf{g}\mathbf{g}^T}$$

$$\mathbf{G} = \begin{bmatrix} g_{-5}\mathbf{g}^T \\ g_{-4}\mathbf{g}^T \\ \vdots \\ g_0\mathbf{g}^T \\ \vdots \\ g_5\mathbf{g}^T \end{bmatrix}$$

- Every row is a scaled version of the same vector

$$R_{-5} = g_{-5}\mathbf{g}^T, \quad R_{-4} = g_{-4}\mathbf{g}^T, \quad \cdots, \quad R_5 = g_5\mathbf{g}^T$$

Only one row is linearly independent, that is $\text{rank}(\mathbf{G}) = 1$

Gaussian Filtering

1. Definitions

F : input image
 H : Gaussian filter kernel
 G : filtered image

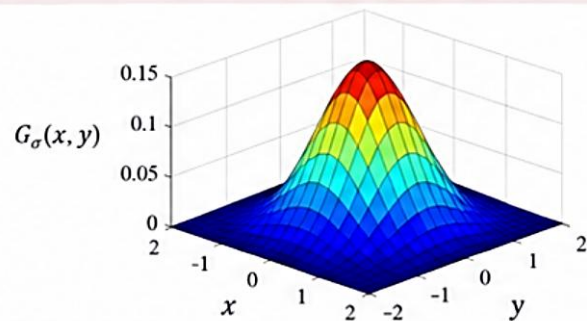
Gaussian filtering performs weighted local averaging.

2. Gaussian kernel

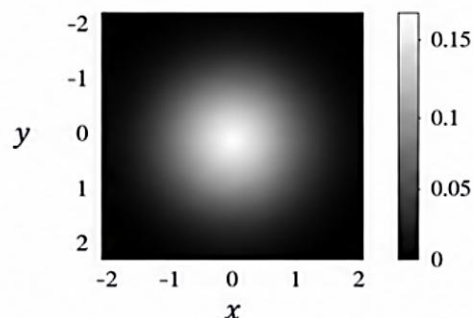
$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Pixels closer to the center receive larger weights.

3. Kernel visualization



(a) 3D Gaussian kernel



(b) 2D Gaussian kernel (top view)

4. Conceptual summary

- Gaussian filtering is a linear smoothing filter.
- It reduces noise while preserving structure better than a mean filter.
- The smoothing strength is controlled by σ .
- Larger σ produces stronger blur.

local image patch

weighted average

output value $G[i, j]$

Note: Gaussian filtering blurs the image, but because nearby pixels are weighted more strongly than distant pixels, it usually preserves edges better than a mean filter.

Mean Filtering

1. Definitions

F : input image

H : mean filter kernel

G : output image

Each output pixel is the average of nearby pixels.

2. Mean filter kernel

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$G[i, j] = \sum_{u=-1}^1 \sum_{v=-1}^1 H[u, v] F[i+u, j+v]$$

For a 3×3 mean filter, sum the 9 neighboring values and divide by 9.

3. Numerical example

Local 3×3 image patch F

0	0	0
0	0	90
0	0	90

$$(0 + 0 + 0 + 0 + 0 + 90 + 0 + 0 + 90) / 9 = 20$$



$$G[i, j] = 20$$

4. Conceptual summary

- Mean filtering is a linear smoothing filter.
- It reduces noise by averaging neighboring pixels.
- It also blurs edges and fine details.
- Equivalent to local averaging (box filtering).

local image patch



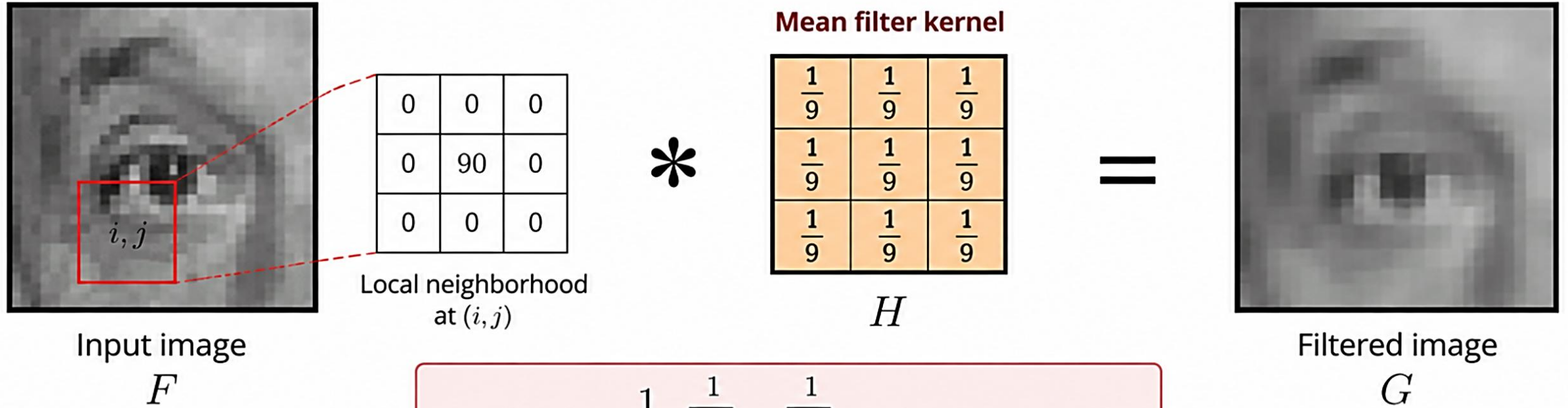
average



output value $G[i, j]$

Note: Mean filtering smooths the image, but strong smoothing can remove edges and fine details.

Mean Filtering



$$G[i, j] = \frac{1}{9} \sum_{u=-1}^1 \sum_{v=-1}^1 F[i+u, j+v]$$

- Each output pixel is the average of the pixels in a local neighborhood.
- Simple and effective for noise reduction (smoothing).
- Equivalent to convolution (or cross-correlation) with a uniform kernel.

Note: Mean filtering blurs the image. It reduces noise but also smooths edges and fine details.

Sharpening Filtering



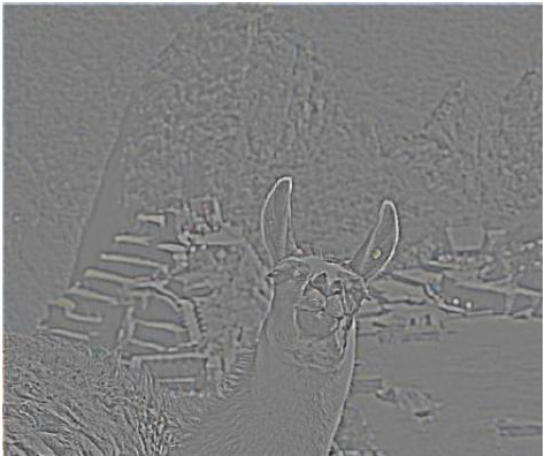
Original

-



Blurred

=



Detailed

+



Original



Detailed

=



Sharpened

Sharpening Filtering (Mean Filtering for Smoothing)



Input image
 F

$$* \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right) =$$



Sharpened image
 G

$$H = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$G[i, j] = \sum_{u=-1}^1 \sum_{v=-1}^1 H[u, v] F[i + u, j + v]$$

- Each output pixel is the average of the pixels in a local neighborhood
- Simple and effective for noise reduction (smoothing)
- Equivalent to convolution (or cross-correlation) with a uniform kernel

Note: Mean filtering blurs the image. It reduces noise but smooths edges and fine details

Sharpening Filtering (Gaussian Filtering for Smoothing)

1. Definitions

F : input image
 H : smoothing kernel (e.g., Gaussian or mean filter)
 Y : sharpened image

Sharpening can be interpreted as adding back image details removed by smoothing.

2. Unsharp masking formula

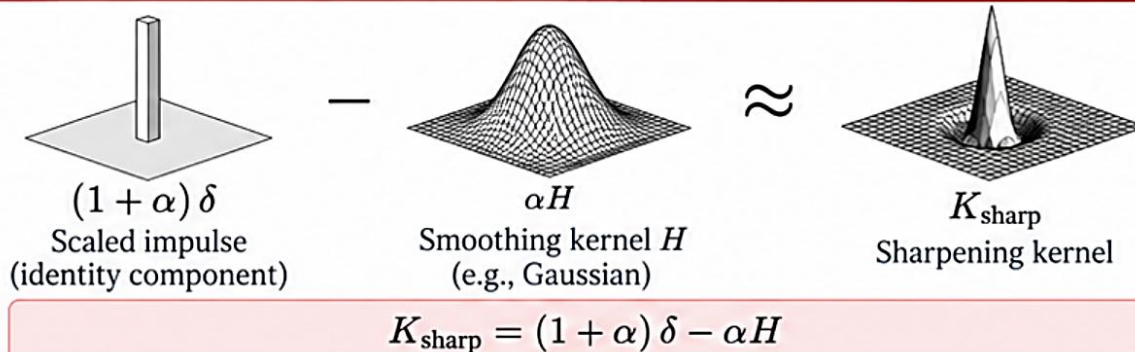
$$Y = F + \alpha \underbrace{(F - F * H)}_{\substack{\text{image details} \\ \text{(high-frequency component)}}$$

$F * H$ is a blurred version of the input image.

$$Y = ((1 + \alpha) \delta - \alpha H) * F$$

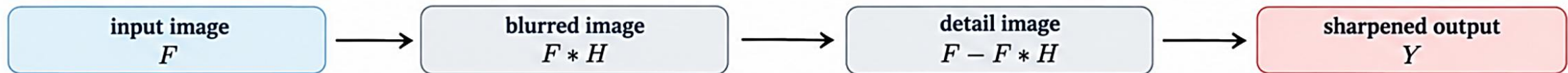
δ : identity (unit impulse) kernel with a single 1 in the center and zeros elsewhere.

3. Kernel interpretation



4. Conceptual summary

- Sharpening emphasizes high-frequency components.
- It enhances edges and fine details.
- Stronger α gives stronger sharpening.
- Excessive sharpening may amplify noise and ringing.



Note: Sharpen filtering strengthens local contrast and edges, but it should be used carefully because noise can also be boosted.